PART B: Cryptography

1.

a. **Write a program in Python, C/C++ or JavaScript to take both the plaintext and the key as its input, then print out the cipher. Assume the plaintext is your name, for the key="THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM" what is the output? Add the screenshots of your program along the input and output and submit it as a PDF file. You should also submit the source code [10 marks].**

Answer:

cypher.cpp U

cypher.cpp > main()

```cpp
#include <iostream>
#include <string>

using namespace std;

// Function to convert a character to its corresponding numeric value
int charToNum(char c) {
    return c - 'A';
}

// Function to convert a numeric value to its corresponding character
char numToChar(int n) {
    return 'A' + (n % 26);
}

// Function to encrypt the plaintext using the key
string encrypt(const string& plaintext, const string& key) {
    string cipher = "";
    for (int i = 0; i < plaintext.length(); ++i) {
        int ptVal = charToNum(plaintext[i]);      // Convert plaintext character to number
        int keyVal = charToNum(key[i]);           // converting key character to number
        int cipherVal = (ptVal + keyVal) % 26;    // Add and mod 26
        cipher += numToChar(cipherVal);           // converting back to character and append to cipher text
    }
    return cipher;
}

int main() {
    // Inputs
    string plaintext = "SANKARSHAN";
    string key = "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM";

    // Encrypt the plaintext
    string cipher = encrypt(plaintext, key);

    // Output the result
    cout << "Plaintext: " << plaintext << endl;
    cout << "Key: " << key.substr(0, plaintext.length()) << endl; // showing only the part of the key that I used
    cout << "Cipher: " << cipher << endl;

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
sankarshanghosh@Sankarshans-MacBook-Pro Midterm % g++ -std=c++11 cypher.cpp -o output
sankarshanghosh@Sankarshans-MacBook-Pro Midterm % ./output
Plaintext: SANKARSHAN
Key: THISISANEX
Cipher: LHVCIJSUEK
```

```
#include <iostream>
#include <string>

using namespace std;

// Function to convert a character to its corresponding numeric value
int charToNum(char c) {
    return c - 'A';
}

// Function to convert a numeric value to its corresponding character
char numToChar(int n) {
    return 'A' + (n % 26);
}

// Function to encrypt the plaintext using the key
string encrypt(const string& plaintext, const string& key) {
    string cipher = "";
    for (int i = 0; i < plaintext.length(); ++i) {
        int ptVal = charToNum(plaintext[i]);      // Convert plaintext character to number
        int keyVal = charToNum(key[i]);           // converting key character to number
        int cipherVal = (ptVal + keyVal) % 26;    // Add and mod 26
        cipher += numToChar(cipherVal);           // converting back to character and append to cipher text
    }
    return cipher;
}

int main() {
    // Inputs
    string plaintext = "SANKARSHAN";
    string key = "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM";

    // Encrypt the plaintext
    string cipher = encrypt(plaintext, key);

    // Output the result
    cout << "Plaintext: " << plaintext << endl;
    cout << "Key: " << key.substr(0, plaintext.length()) << endl; // showing only the part of the key that I used
    cout << "Cipher: " << cipher << endl;

    return 0;
}
```

b. **Explain how one can decrypt the encrypted message using the encryption algorithm in part (a)? Write a program, in JavaScript, C/C++ or Python to take both the plaintext and the key as its input, then print out the plaintext. Test it using the results of the previous program. Add the screenshots of your program along the input and output and submit it as a PDF file. You should also submit the source code [10 marks].**

Answer:

The whole process is like reversing the steps we took to encrypt it. When Alice and Bob encrypted the message, they used a key and the message to make a new cypher. They did this by adding up the values of corresponding letters from the message and the key, then taking the result modulo 26 to get the encrypted letters.

To decrypt this cipher, Bob will need the same key that was used to encrypt it. Here's the simple method to get the original message back:

- Convert each letter of the cipher and the key back to numbers.
- Subtract the key number from the cipher number for each letter.
- Handle negatives and wrap around using modulo 26.
- Convert the numbers back to letters.

It's really just doing the encryption process backward.

cypher.cpp U decypher.cpp U

decypher.cpp > main()

```cpp
#include <iostream>
#include <string>

using namespace std;

// Function to convert a character to its corresponding numeric value
int charToNum(char c) {
    return c - 'A';
}

// Function to convert a numeric value to its corresponding character
char numToChar(int n) {
    // Ensure the result is positive by adding 26 and taking modulo 26
    return 'A' + ((n + 26) % 26);
}

// Function to decrypt the cipher text using the key
string decrypt(const string& cipher, const string& key) {
    string plaintext = "";
    for (int i = 0; i < cipher.length(); ++i) {
        int cipherVal = charToNum(cipher[i]);     // Convert cipher character to number
        int keyVal = charToNum(key[i]);           // Convert key character to number
        int plainVal = (cipherVal - keyVal) % 26; // Subtract and mod 26
        plaintext += numToChar(plainVal);         // Convert back to character and append to plaintext
    }
    return plaintext;
}

int main() {
    // Inputs: example cipher text and key
    string cipher = "LHVCIJSUEK"; // got this by running the previous program
    string key = "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM";

    // Decrypt the cipher text
    string plaintext = decrypt(cipher, key);

    // Output the results
    cout << "Cipher: " << cipher << endl;
    cout << "Key: " << key.substr(0, cipher.length()) << endl; // Show only the part of the key used
    cout << "Plaintext: " << plaintext << endl;

    return 0;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
sankarshanghosh@Sankarshans-MacBook-Pro Midterm % g++ -std=c++11 cypher.cpp -o output
sankarshanghosh@Sankarshans-MacBook-Pro Midterm % ./output
Plaintext: SANKARSHAN
Key: THISISANEX
Cipher: LHVCIJSUEK
sankarshanghosh@Sankarshans-MacBook-Pro Midterm % g++ -std=c++11 decypher.cpp -o decypheredOutput
sankarshanghosh@Sankarshans-MacBook-Pro Midterm % ./decypheredOutput
Cipher: LHVCIJSUEK
Key: THISISANEX
Plaintext: SANKARSHAN
```

```
#include <iostream>

#include <string>

using namespace std;

// Function to convert a character to its corresponding numeric value

int charToNum(char c) {

    return c - 'A';

}

// Function to convert a numeric value to its corresponding character

char numToChar(int n) {

    // Ensure the result is positive by adding 26 and taking modulo 26

    return 'A' + ((n + 26) % 26);

}

// Function to decrypt the cipher text using the key

string decrypt(const string& cipher, const string& key) {

    string plaintext = "";

    for (int i = 0; i < cipher.length(); ++i) {

        int cipherVal = charToNum(cipher[i]);     // Convert cipher character to number

        int keyVal = charToNum(key[i]);           // Convert key character to number

        int plainVal = (cipherVal - keyVal) % 26; // Subtract and mod 26

        plaintext += numToChar(plainVal);         // Convert back to character and append to plaintext

    }

    return plaintext;

}

int main() {

    // Inputs: example cipher text and key

    string cipher = "LHVCIJSUEK"; // got this by running the previous program

    string key = "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM";

    // Decrypt the cipher text

    string plaintext = decrypt(cipher, key);
```

```
    // Output the results

    cout << "Cipher: " << cipher << endl;

    cout << "Key: " << key.substr(0, cipher.length()) << endl; // Show only the part of the key used

    cout << "Plaintext: " << plaintext << endl;

    return 0;

}
```

c. Under what conditions (specify two) can a cipher be broken, meaning that an attacker can decipher the encrypted message without having the key? Support your reasoning with example(s) [6 marks].

Answer:

Here are two conditions under which a cipher can be broken even if the attacker doesn't have the key:

1. **Short or Repeated Keys:** If the encryption key is short or repeats a lot, it makes the cipher vulnerable to attacks. For example, if the key is "ABC" and the plaintext is much longer, like "HELLOWORLDHELLOWORLD", then the key repeats over and over ("ABCABCABCABCABC..."). An attacker can analyze patterns in the encrypted message and match them to patterns that are known to occur in the language of the plaintext (like English). This method is called frequency analysis and it exploits the fact that in any language, certain letters and combinations appear more frequently than others. If the key is repeating, these patterns will start showing up in the ciphertext.
2. **Known Plaintext Attack:** If an attacker somehow gets access to both the plaintext (the original message) and the ciphertext (the encrypted message), they can work out what the key must be. This is almost like reverse engineering. This could happen if the same key is used over and over again to encrypt different messages and somehow the attacker manages to get one pair of plaintext and ciphertext. Once they have the key from this pair, they can decrypt any other messages encrypted with the same key. For instance, if someone knows that the word "HELLO" encrypts to "ZINCS" using a certain key, they can find out the key by reversing the encryption process. After that, any other message encrypted with the same key can be decrypted easily.

These situations show why it's important to use long, non-repeating keys and to change keys regularly to keep the encrypted data safe from all these attacks.

d. Identify THREE key factors for defining a robust key to enhance algorithm security? Explain your reasons [6 marks].

Answer:

When we talk about making a key strong for better security in encryption, some three important things to keep in mind will be:

1. **Length of the Key:** A longer key means more security. If you have a short key, it's easier for someone trying to break the code to guess it because there aren't many combinations. But if the key is long, then the number of possible combinations becomes huge. This makes guessing the key super hard. So, always go for a longer key when setting up encryption.
2. **Complexity of the Key:** The key shouldn't be simple or something easy to guess like "ABC123" or your birthday. A good key should have a mix of different types of characters—letters in uppercase and lowercase, numbers, and special symbols. This makes the key complex and reduces the chances of someone else guessing it. For example, a key like "Fg#47@bQ" is much tougher to crack than something simple like "hello".
3. **Randomness:** The key should be random, meaning it shouldn't have any predictable pattern or sequence that makes it easy to guess. If a key is truly random, it doesn't follow any logical order that someone could track down. This randomness adds an extra layer of security because it removes logical ways of breaking the cipher.

These factors make the key strong and tough for anyone to crack, keeping your encrypted data safe.

2.

   a. **What is the purpose of padding in cryptography algorithms such as DES? Explain it using a simple example [6 marks].**
      Answer:

Padding in cryptography, especially in block ciphers like DES, is used to ensure that the plaintext data can be evenly divided into fixed-sized blocks that the cipher can process. Since DES processes data in 64-bit blocks, if the length of the data isn't a multiple of 64 bits, padding is added to make up the difference. This prevents data corruption and ensures that every block is filled to prevent leakage of information about the plaintext data's length, which might help attackers.

For example, if you have a message "HEY" to encrypt using DES, and you convert each character to binary, it won't make up a full 64-bit block. Padding is added to extend this to a complete block size that DES can encrypt.

b. **We know that the size of the key in DES is 64 bits, assume for padding we just add 0 at the end of the plaintext (the original message), what is the result of padding for the following text? At first replace the “YYY” with the first 3 letters of your name or family then calculate the padding (show the result in hexadecimal format): “YYYComputerSecurity”. Show your answer step by step [6 marks].**
   Answer:

Let's say "YYY" is replaced by "SAN", making the plaintext "SANComputerSecurity". This plaintext needs to be padded to match the 64-bit block size required by DES.

1. Convert "SANComputerSecurity" to ASCII in hexadecimal: "53 41 4E 43 6F 6D 70 75 74 65 72 53 65 63 75 72 69 74 79".
2. Calculate the length in bits: 19 characters x 8 bits = 152 bits.
3. Since DES needs 64-bit blocks, calculate padding: 64 x 3 = 192 bits (three blocks) - 152 bits = 40 bits of padding needed.
4. In hexadecimal, zero padding of 40 bits would be added as zeroes: "00 00 00 00 00".

Final hexadecimal representation with padding might look like: "53 41 4E 43 6F 6D 70 75 74 65 72 53 65 63 75 72 69 74 79 00 00 00 00 00".

c. **We know that RSA works with numbers. Assume we want to encrypt the plaintext M=“ComputerSecurity”. We know we should use the following formula: C=Me mod N in which (e,N) is the public key. But before using this formula, at first, we should convert M into a number. To do so, there are different solutions. Search the Internet and explain one of them. Show different steps of the selected solution on the given plaintext M=“ComputerSecurity” as an example. Consider padding as well [10 marks].**
   Answer:

RSA requires the plaintext to be represented as a number before applying the encryption formula. One common method to achieve this is by converting each character of "ComputerSecurity" to its ASCII equivalent and then concatenating these to form a long number.

Steps:

1. Convert each character to its ASCII value.
2. Concatenate these values to form a number.

For "ComputerSecurity", the ASCII values are:

- C = 67, o = 111, m = 109, p = 112, u = 117, t = 116, e = 101, r = 114, S = 83, e = 101, c = 99, u = 117, r = 114, i = 105, t = 116, y = 121.

Concatenating these values would result in a very large number, which RSA can then encrypt using the formula C=Memod NC = M^e \mod NC=MemodN, where (e,N)(e,N)(e,N) is the public key.

d. What is a digital signature? Is it possible to generate a digital signature using RSA? What about DES? Explain your reasoning [6 marks].

Answer:

A digital signature is a way to verify if a message is authentic and the integrity of a message, software, or digital document. It is essentially the digital equivalent of a handwritten signature or stamped seal, but obviously offers far more security.

RSA can be used for digital signatures by applying the RSA algorithm to hash values of data, which are then encrypted with a private key. Anyone with its public key can verify the authenticity by decrypting the signature and comparing the hash values.

DES on the other hand is a symmetric key cipher, meaning the same key is used for both encryption and decryption. This symmetry makes DES unsuitable for digital signatures, as it cannot provide the public/private key mechanism needed to verify signatures without revealing the key used to create them.

RSA is commonly used for digital signatures, while DES is not used due to its symmetric nature and limitations in securely managing key distribution for such purposes.